



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Отчет по выполнению практического задания №6.1

Тема: Быстрый доступ к данным с помощью хеш-таблиц

Дисциплина: структуры и алгоритмы обработки данных

Выполнил студент
группа

Мижурин А.А.
ИКБО-50-25

Москва 2026

ЦЕЛЬ РАБОТЫ

Цель работы: освоить приёмы хеширования и эффективного поиска элементов динамического множества.

В работе используется **вариант 13**. Метод разрешения коллизий — открытая адресация с двойным хешированием. Полезные данные записи включают семизначный номер банковского счёта, ФИО владельца и адрес.

1 ХОД РАБОТЫ

1.1 Формулировка задачи

Разработать приложение, использующее хеш-таблицу для прямого доступа к элементам динамического множества. Множество реализовать на массиве. Создать класс с операциями вставки, удаления, поиска по ключу и вывода. Включить в класс массив полезных данных и хеш-таблицу. Реализовать автоматическое расширение таблицы и рехеширование, начальное заполнение 5–7 записями и текстовый командный интерфейс. Проверить все базовые операции, изменение размера и работу с коллизиями.

В работе используется **вариант 13**. Способ разрешения коллизий — открытая адресация с двойным хешированием. Ключ записи — семизначный целый номер банковского счёта. Полезные данные — ФИО и адрес владельца.

1.2 Математическая модель решения

1.2.1 Массив данных и хеш-таблица

Полезные данные хранятся в динамическом массиве `data_`. Элемент массива содержит номер банковского счёта, ФИО и адрес. Удалённая позиция освобождается и затем может быть использована для новой записи.

Отдельный массив `slots_` представляет хеш-таблицу. Его занятая ячейка хранит пару «ключ — индекс»: номер счёта и индекс соответствующей записи в `data_`. Поэтому после нахождения ключа полная запись сразу извлекается из массива полезных данных по индексу.

Ячейка хеш-таблицы может находиться в одном из трёх состояний:

- `Empty` — ячейка ещё не использовалась;
- `Occupied` — ячейка содержит ключ и индекс записи;
- `Deleted` — запись удалена, но через ячейку может продолжаться цепочка пробирования.

1.2.2 Двойное хеширование

Пусть m — размер хеш-таблицы, а k — номер счёта. Начальная ячейка и шаг пробирования вычисляются двумя функциями:

$$h_1(k) = k \bmod m,$$
$$h_2(k) = 1 + (k \bmod (m - 1)).$$

На попытке с номером i проверяется ячейка

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m, \quad i = 0, 1, \dots, m - 1.$$

Размер таблицы выбирается простым числом. Шаг принимает значения от 1 до $m - 1$ и позволяет последовательно проверить все ячейки. Если два ключа получают одинаковую начальную позицию, следующая позиция определяется второй хеш-функцией. Так разрешается коллизия.

1.2.3 Поиск

Поиск начинается с ячейки $h_1(k)$. На каждой попытке вычисляется следующая позиция по формуле двойного хеширования. Если занятая ячейка содержит искомый ключ, из неё берётся индекс записи. Ячейка Deleted пропускается. Встреча Empty означает, что искомого ключа в цепочке нет. Если пустая ячейка не встретилась, проверяется не более m позиций.

1.2.4 Вставка

Сначала выполняется поиск ключа, чтобы не допустить повторного номера счёта. Для полезных данных используется свободная позиция массива `data_` или добавляется новый элемент. Затем для ключа строится последовательность проб. Запоминается первая удалённая ячейка. Запись помещается в неё либо в первую пустую ячейку. В хеш-таблицу записываются ключ и индекс полезных данных.

Перед вставкой рассчитывается доля занятых и удалённых ячеек. Если после добавления она превысит 65 %, размер хеш-таблицы увеличивается. Новым размером становится ближайшее простое число, не меньшее удвоенного прежнего размера.

1.2.5 Удаление

По ключу находится ячейка хеш-таблицы и связанная с ней позиция массива данных. Полезная запись освобождается, а её индекс сохраняется для повторного использования. Ячейка хеш-таблицы получает состояние Deleted. Её нельзя сразу сделать пустой, поскольку это преждевременно остановило бы поиск других ключей, размещённых дальше из-за коллизии.

1.2.6 Расширение и рехеширование

После изменения размера результаты обеих хеш-функций меняются. Поэтому создаётся новая хеш-таблица, а все действующие пары «ключ — индекс» вставляются в неё повторно. Сам массив полезных данных не перестраивается: индексы записей в нём остаются прежними. Удалённые ячейки в новую таблицу не переносятся.

1.3 Код программы с комментариями

Запись варианта 13 представлена структурой BankAccount. В классе HashTable отдельно объявлены массив полезных данных и массив ячеек хеш-таблицы. На рисунке 1.1 показаны полные объявления BankAccount и HashTable, включая открытые операции, внутреннюю структуру Slot, массивы и служебные методы. Объявление `using std::size_t;` позволяет далее писать короткое имя `size_t`; типы `std::string` и `std::vector` оставлены с указанием пространства имён.

bank_account.hpp

```
// Полезные данные одной записи варианта 13.
struct BankAccount {
    int accountNumber{};
    std::string fullName;
    std::string address;
};
```

hash_table.hpp

```
using std::size_t;
class HashTable {
public:
    explicit HashTable(size_t initialCapacity = 7);

    bool insert(const BankAccount &account);
    bool erase(int accountNumber);
    BankAccount *find(int accountNumber);
    const BankAccount *find(int accountNumber) const;

    std::vector<BankAccount> values() const;
    size_t size() const;
    size_t capacity() const;
    double loadFactor() const;

private:
    enum class SlotState { Empty, Occupied, Deleted };

    struct Slot {
        int key{};
        size_t dataIndex{};
        SlotState state{SlotState::Empty};
    };

    static constexpr double maxLoadFactor_ = 0.65;

    // Полезные данные и хеш-таблица хранятся в отдельных массивах.
    std::vector<std::optional<BankAccount>> data_;
    std::vector<size_t> freeDataIndices_;
    std::vector<Slot> slots_;
    size_t size_{0};
    size_t deletedCount_{0};

    static bool isPrime(size_t number);
    static size_t nextPrime(size_t number);
    size_t primaryHash(int key) const;
    size_t secondaryHash(int key) const;
    size_t probeIndex(int key, size_t attempt) const;
    size_t findIndex(int key) const;
    void rehash(size_t requestedCapacity);
    void insertWithoutResize(int key, size_t dataIndex);
};
```

Рисунок 1.1 – Полные объявления структуры BankAccount и класса HashTable

Две хеш-функции и номер очередной проверяемой ячейки реализованы отдельными методами:

Листинг 1.1 – Вычисление позиций методом двойного хеширования

```
size_t HashTable::primaryHash(int key) const {
    return static_cast<size_t>(key) % slots_.size();
}

size_t HashTable::secondaryHash(int key) const {
    // При простом размере таблицы шаг от 1 до m - 1 взаимно прост с m.
    return 1 + static_cast<size_t>(key) % (slots_.size() - 1);
}

size_t HashTable::probeIndex(int key, size_t attempt) const {
    // Формула двойного хеширования:  $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$ .
    return (primaryHash(key) + attempt * secondaryHash(key)) % slots_.size();
}
```

Метод поиска останавливается только при нахождении ключа, встрече пустой ячейки или после проверки всего массива:

Листинг 1.2 – Поиск ячейки по номеру счёта

```
size_t HashTable::findIndex(int key) const {
    for (size_t attempt = 0; attempt < slots_.size(); ++attempt) {
        const size_t index = probeIndex(key, attempt);
        const Slot &slot = slots_[index];

        // Пустая ячейка завершает цепочку пробирования.
        if (slot.state == SlotState::Empty) {
            return slots_.size();
        }
        if (slot.state == SlotState::Occupied && slot.key == key) {
            return index;
        }
    }
    return slots_.size();
}
```

При вставке сначала проверяется уникальность ключа и необходимость расширения. Затем запись размещается в свободной позиции массива данных, а её ключ и индекс — в хеш-таблице.

Листинг 1.3 – Вставка полезных данных и хеш-записи

```
bool HashTable::insert(const BankAccount &account) {
    if (findIndex(account.accountNumber) != slots_.size()) {
        return false;
    }

    // Учитываем и удалённые ячейки: их избыток удлинит поиск.
    const double projectedOccupancy =
        static_cast<double>(size_ + deletedCount_ + 1) / slots_.size();
    if (projectedOccupancy > maxLoadFactor_) {
        rehash(slots_.size() * 2);
    }

    size_t dataIndex;
    if (freeDataIndices_.empty()) {
        dataIndex = data_.size();
        data_.push_back(account);
    } else {
        dataIndex = freeDataIndices_.back();
        freeDataIndices_.pop_back();
        data_[dataIndex] = account;
    }

    insertWithoutResize(account.accountNumber, dataIndex);
    return true;
}
```

Удаление освобождает полезную запись, но сохраняет непрерывность цепочки пробирования с помощью состояния Deleted:

Листинг 1.4 – Удаление записи

```
bool HashTable::erase(int accountNumber) {
    const size_t index = findIndex(accountNumber);
    if (index == slots_.size()) {
        return false;
    }

    // Освобождаем запись и сохраняем цепочку пробирования.
    data_[slots_[index].dataIndex].reset();
    freeDataIndices_.push_back(slots_[index].dataIndex);
    slots_[index].state = SlotState::Deleted;
    --size_;
    ++deletedCount_;
    return true;
}
```

При рехешировании размер выбирается методом nextPrime. Действующие ключи получают новые позиции, но продолжают ссылаться на те же элементы data_:

Листинг 1.5 – Расширение и рехеширование

```
void HashTable::rehash(size_t requestedCapacity) {
    std::vector<Slot> oldSlots = std::move(slots_);
    slots_ = std::vector<Slot>(nextPrime(requestedCapacity));
    size_ = 0;
    deletedCount_ = 0;

    // Индексы хеш-таблицы зависят от её размера.
    for (const Slot &slot : oldSlots) {
        if (slot.state == SlotState::Occupied) {
            insertWithoutResize(slot.key, slot.dataIndex);
        }
    }
}
```

При запуске таблица автоматически заполняется семью записями. Текстовый интерфейс предоставляет команды добавить, найти, удалить, вывести, статистика, помощь и выход. ФИО и адрес отделяются от номера счёта точками с запятой.

Листинг 1.6 – Автоматическое заполнение и команды интерфейса

```
void fillTable(HashTable &table) {
    for (const BankAccount &account : initialAccounts()) {
        table.insert(account);
    }
}

void printHelp() {
    std::cout << "Команды:\n"
        << "  добавить <номер>;<ФИО>;<адрес>\n"
        << "  найти <номер>\n"
        << "  удалить <номер>\n"
        << "  вывести\n"
        << "  статистика\n"
        << "  помощь\n"
        << "  выход\n";
}
```

1.4 Результаты тестирования

Сначала выполнен демонстрационный сценарий: автоматическое заполнение, поиск счёта 1000007, его удаление, повторный поиск и вставка нового счёта 6789012.

```

Демонстрация варианта 13
Записей: 7, размер массива: 17, заполнение: 41.18%
Поиск счёта 1000007: 1000007 | Петров Пётр Петрович |
Москва, ул. Арбат, д. 12
Удаление счёта 1000007: выполнено
Повторный поиск: запись не найдена
Вставка счёта 6789012: выполнена
Записей: 7, размер массива: 17, заполнение: 41.18%

```

Рисунок 1.2 – Результат демонстрационного сценария

После добавления семи начальных записей хеш-таблица расширилась с 7 до 17 ячеек. Поиск существующего счёта вернул правильные ФИО и адрес. После удаления повторный поиск сообщил об отсутствии записи. Новая запись была успешно добавлена.

Для проверки коллизий использованы ключи 1000000, 1000007 и 1000014. При размере таблицы 7 они имеют одинаковую начальную позицию. Проверены вставка и поиск всех трёх записей, а также удаление первого элемента цепочки с последующим поиском остальных элементов.

Таблица 1.1 – Результаты автоматического тестирования

№	Проверяемое действие	Результат
1	Вставка обычной записи	Успех
2–3	Вставка двух записей с коллизией	Успех
4	Проверка количества записей	Успех
5	Поиск записи с коллизией	Успех
6	Поиск отсутствующего ключа	Успех
7	Отказ от повторного ключа	Успех
8–10	Удаление и поиск за удалённой ячейкой	Успех
11	Удаление отсутствующего ключа	Успех
12	Повторное использование свободной позиции	Успех
13–14	Расширение и сохранение записей	Успех
15	Вывод всех действующих записей	Успех

```

[УСПЕХ] вставка записи с коллизией
[УСПЕХ] вставка второй записи с коллизией
[УСПЕХ] поиск записи, вызвавшей коллизию
[УСПЕХ] удаление первой записи цепочки
[УСПЕХ] поиск за удалённой ячейкой остаётся корректным
[УСПЕХ] автоматическое расширение массива
[УСПЕХ] сохранность записей после рехеширования
[УСПЕХ] вывод содержит все занятые записи
Пройдено проверок: 15 из 15

```

Рисунок 1.3 – Итог автоматического тестирования

ВЫВОД

В практической работе созданы массив записей банковских счетов и отдельная хеш-таблица варианта 13. Реализованы вставка, удаление, поиск по номеру счёта, вывод записей, автоматическое заполнение, расширение массива хеш-таблицы и рехеширование. Для разрешения коллизий применена открытая адресация с двойным хешированием.

При начальном размере 7 ячеек автоматическая вставка семи записей привела к расширению хеш-таблицы до 17 ячеек. В демонстрационном запуске поиск, удаление и повторная вставка выполнены правильно. Все 15 автоматических проверок, включая работу с коллизиями и сохранение записей после рехеширования, завершились успешно. Задача решена, цель работы достигнута.